

Part 2: Definició de conceptes teòrics

1. Què és una prova unitària?

Una **prova unitària** és una prova automatitzada que verifica el funcionament correcte d'una unitat mínima de codi, normalment un mètode o una classe, de manera aïllada.

L'objectiu és comprovar que cada part del programa funciona correctament de forma independent.

2. Quins avantatges té fer proves unitàries durant el desenvolupament?

- Detecten errors ràpidament.
- Faciliten el manteniment del codi.
- Permeten modificar el codi amb més seguretat.
- Serveixen com a documentació del comportament esperat.
- Milloren la qualitat del programari.
- Faciliten el treball en equip.

3. Què és JUnit? Quina versió esteu fent servir?

JUnit és un framework per fer proves unitàries en aplicacions Java.

Permet:

- Crear mètodes de prova.
- Utilitzar assertions.
- Executar proves automàticament.
- Veure quines proves passen i quines fallen.

Actualment, la versió més utilitzada és **JUnit 5** que és la que normalment crea NetBeans si el projecte és modern Maven o Ant actualitzat.

4. Què fa una "assertion"? Posa'n un exemple.

Una **assertion** comprova que el resultat obtingut és igual al resultat esperat.

Si no coincideixen, la prova falla.

Exemple:

```
assertEquals(5, calculadora.suma(2, 3));
```

Aquesta assertion comprova que $2 + 3$ sigui igual a 5.

5. Quina diferència hi ha entre una prova que passa i una que falla?

- **Prova que passa:** El resultat obtingut és el que s'esperava.
- **Prova que falla:** El resultat és diferent de l'esperat o es produeix una excepció inesperada.

6. Com hauries de nomenar els mètodes de prova per fer que siguin clars i entenedors?

Han de descriure:

- El mètode que es prova
- La situació
- El resultat esperat

Exemples correctes:

```
testSumaPositiu()  
testDivisioPerZeroLlançaExcepcio()  
testMultiplicacioNegatiu()
```

També es pot usar el format:

```
metode_condicio_resultatEsperat
```

Exemple:

```
suma_dosPositius_retornCorrecte
```

Reflexió final

1. Què passa quan una prova falla? Com ho detectes a NetBeans?

Quan una prova falla:

- Apareix en vermell.
- NetBeans mostra un missatge d'error.
- Indica quina assertion ha fallat.
- Mostra el valor esperat i el valor obtingut.

Això permet identificar ràpidament el problema.

2. Què faries si una prova falla però creus que el codi està bé?

- Revisaria la prova per comprovar que l'assertion sigui correcta.
- Verificaria els valors esperats.
- Comprovaria si el comportament definit és el que realment es vol.
- Faria proves manuals si cal.

Pot ser que l'error estigui a la prova i no al codi.

3. Creus que és útil fer primer les proves i després escriure el codi? Per què?

Sí. Això s'anomena **TDD (Test Driven Development)**.

Avantatges:

- Obliga a pensar abans en el comportament.
- Fa el codi més modular.
- Redueix errors.
- Millora el disseny del programa.